



18BIG

Big Data Tools & Architecture

Lecture 03 — Storage Formats for the Lakehouse

www.bighub.ai

Your guides to applied AI



■ Recap: last time

Three questions from **Session 02 — Data Sources & Bronze Ingestion**:

1. What belongs in **Bronze**, and what do we deliberately **not** do there?
2. What is a **watermark**, and why can't it see **hard deletes**?
3. Why do we design every ingestion job around *"what if this runs twice?"* — what's that property called?

Warm-up — answer out loud before we build on it.

■ Recap: answers

1. **Bronze** keeps data *as it arrived* (raw, duplicates and bad rows included) + lineage metadata. We **don't** clean, deduplicate, reshape or enforce a schema — that's Silver.
2. A **watermark** is the last value loaded (e.g. `max(order_id)`); the next run reads only rows above it. A **hard delete** removes the row, so nothing rises above the watermark → it's invisible (needs a full compare or CDC — Change Data Capture).
3. **Idempotence** — re-running must not duplicate or corrupt data, so retries, backfills and replays are safe.

■ Today

- **Row vs. columnar** — the core idea
- **Avro** — the row-oriented format
- **Parquet** — the columnar format (anatomy, compression)
- **Avro vs Parquet** — pick & combine
- **Pushdown** — projection + predicate
- **Partitioning** — folders & pruning, the small-files trap
- **File vs. table format** — the road to Delta (Session 4)
- **Hands-on** — a measurement lab: *prove* pruning helps



Part 1

Row vs. columnar storage

Your guides to applied AI



■ The question behind the format

Analytics asks **narrow, deep** questions: *"total revenue last month"* touches **2 columns** across **millions of rows** — not whole rows.

Row storage — OLTP

On**L**ine **T**ransaction **P**rocessing — the live app: orders, payments

- Row stays together on disk
- Great to **fetch / update one record**
- A column scan reads **every byte** of every row

Columnar storage — OLAP

On**L**ine **A**nalytical **P**rocessing — reporting & analytics over history

- Each column stored **contiguously**
- Read **only the columns you select**
- Similar values sit together → **compress hard**

Two layouts, two jobs — the rest of today is the formats that live in each camp

■ Same table, two layouts

THE TABLE — LOGICAL ROWS YOU WRITE & QUERY

id	country	amount
1	CZ	120
2	DE	80
3	CZ	200

ROW-MAJOR (OLTP) — EACH ROW TOGETHER

1	CZ	120	2	DE	80	3	CZ	200
---	----	-----	---	----	----	---	----	-----

COLUMNAR (OLAP) — EACH COLUMN TOGETHER

1	2	3	CZ	DE	CZ	120	80	200
---	---	---	----	----	----	-----	----	-----

Same three rows, same values — only the order on disk changes

- **A query reads only what it needs**

`SELECT sum(amount) FROM orders` — dimmed cells are **bytes never read**.

ROW-MAJOR — WALK EVERY ROW TO REACH EACH AMOUNT

1	CZ	120	2	DE	80	3	CZ	200
---	----	-----	---	----	----	---	----	-----

COLUMNAR — SEEK STRAIGHT TO THE AMOUNT COLUMN

1	2	3	CZ	DE	CZ	120	80	200
---	---	---	----	----	----	-----	----	-----

A real table has 50+ columns — a 3-column query reads ~6% of the bytes, not 100%

■ Each format has a home

Row store → OLTP

- Whole **records** read/written fast
- Insert / update / delete **one row** cheaply
- Serves many small **transactions**
- Powers **checkout, banking, CRM**
- PostgreSQL · MySQL · Oracle

Columnar → OLAP

- **Scans + aggregates** over a few columns, many rows
- Compresses hard; poor at single-row writes
- Serves **analytics**, not transactions
- Powers **dashboards, reporting, the lakehouse**
- Vertica · ClickHouse · BigQuery · Snowflake

Rule of thumb: **many small transactions → rows; big analytical scans → columns.**

■ In practice: a logistics network

Operational · row store (OLTP)



Capture every parcel event, live

INSERT scan

UPDATE status

where is #A1234?



nightly / streaming load

Analytics · columnar store (OLAP)



Aggregate millions of shipments for the business

on-time % by region

avg transit time

volume YoY

Same shipments, two stores: **row-oriented** absorbs the live firehose of writes; **columnar** answers the big analytical questions.



Part 2

Avro — the row-oriented format

Your guides to applied AI



■ What is Avro?

- **Row-oriented** binary serialization format
- Born in the **Hadoop** project (2009)
- A record's fields are stored **together**, row after row
- The **write-side** counterpart to columnar formats

Built for

- **Fast writes** & whole-record reads
- **Evolving schemas** without breaking readers
- **Moving records** between systems (Kafka, RPC)

Columnar formats are built to *read* slices of huge tables; Avro is built to *move and append* whole records.

■ Inside an Avro file

0 b j · 1 - m a g i c b y t e s

File header — writer schema (JSON) · codec · 16-byte sync marker

DATA BLOCK 0

record · record · record ... → sync marker

DATA BLOCK 1 — APPEND MORE

record · record ... → sync marker

The **schema sits at the front** — write it once, then stream blocks of whole records; no need to scan to the end to learn the layout.

■ Schema & evolution, by example

The record schema (JSON)

```
{
  "type": "record",
  "name": "ParcelScan",
  "fields": [
    {
      "name": "parcel_id",
      "type": "string"
    },
    {
      "name": "status",
      "type": "string"
    },
    {
      "name": "scanned_at",
      "type": "long"
    }
  ]
}
```

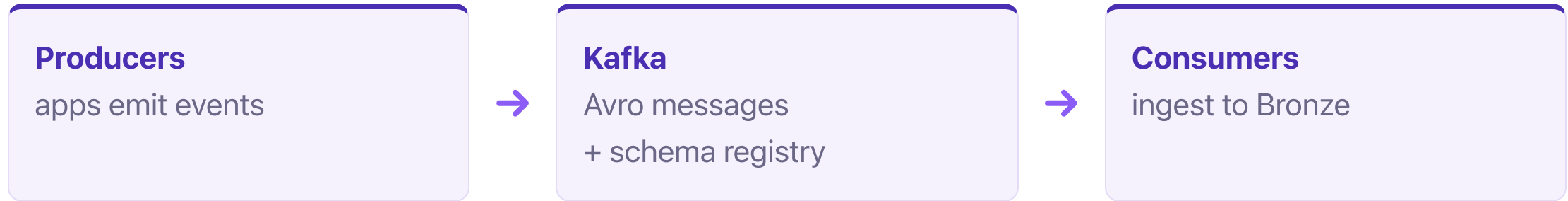
Add a field, safely

```
{
  "name": "weight_kg",
  "type": ["null", "double"],
  "default": null
}
```

- Old readers **ignore** it
- New readers get **null** for old records

Schema evolution: producers & consumers can run different versions and still talk

■ Where Avro shines



Great fit

- **Event streaming** — Kafka's default payload
- **CDC** & message queues
- Row-by-row **ingestion** into the lake

Not its job

- **Analytical scans** over a few columns
- Column-level compression / column skipping
- → that's a job for **columnar formats**



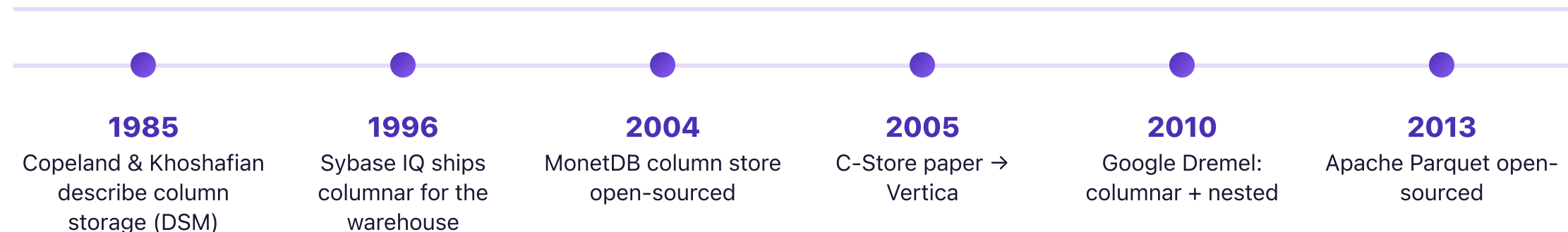
Part 3

Parquet — the columnar format

Your guides to applied AI



■ Columnar is a 40-year-old idea



DSM = Decomposition Storage Model — the 1985 idea of storing each column separately, the root of every columnar engine since.

The lakehouse didn't invent columnar — it brought a **40-year database idea to open files on cheap object storage.**

■ Where Parquet came from

- **2010** — Google's **Dremel** paper: columnar layout + *record shredding* for nested data
- **2012–13** — **Twitter + Cloudera** build an open columnar format for Hadoop
- Named after **parquet flooring** — repeating, interlocking blocks
- **Mar 2013** open-sourced · **2015** top-level Apache project

Led by Julien Le Dem (Twitter) & Nong Li (Cloudera); the Dremel "striping & assembly" (repetition/definition levels) is still how Parquet encodes nested data.

Why they built it

- Twitter: fast analytics on a huge Hadoop cluster
- Cloudera: a columnar format for **Impala**
- Became the **default** for Spark, Hive, Impala, Presto, Drill

■ Why columns compress better

Same column, same kind of data

- A `country` column is a handful of repeated strings
- An `order_ts` column is near-sorted numbers
- Neighbours are **similar** → tiny after encoding

Encodings do the work

- **Dictionary** — store each distinct value once, rows become small ints
- **RLE** (Run-Length Encoding) — "CZ × 4 000" instead of 4 000 copies
- Then **Snappy / Zstd** on top

Row storage mixes an int, a string and a timestamp on every row — far less for a compressor to exploit.

■ The encodings, by example

Dictionary — store each distinct value once, rows become small codes

CZ CZ DE CZ DE CZ → dict: 0=CZ 1=DE + 0 0 1 0 1 0

RLE — Run-Length Encoding — collapse a run of repeats into value × count

0 0 0 0 1 1 → 0 ×4 , 1 ×2

Bit-packing — use only as many bits as the value range needs

codes are 0 / 1 → 1 bit each instead of int32 = 32 bits

Parquet applies **RLE + bit-packing** to dictionary codes, then a codec (**Snappy / Zstd**) on top — three wins stacked.

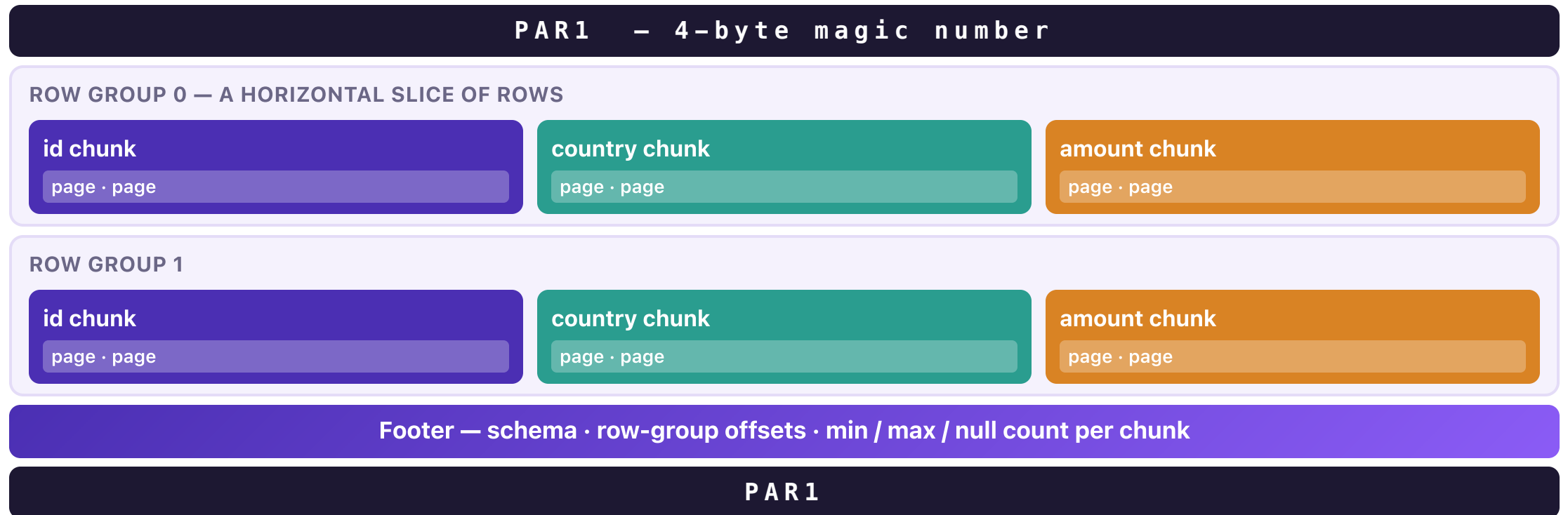
■ The anatomy



- A **footer** at the end holds the schema and **per-row-group statistics** (min / max / null counts).
- A **PAX** (Partition Attributes Across) layout: row groups split the work across workers; columns inside each group give pruning + compression.

Nesting built on Google's Dremel paper; the open Apache Parquet format since 2013 (next slide: the same file as bytes on disk).

■ A Parquet file on disk



Metadata is written **last** → single-pass write; readers seek the **footer first** to decide what to skip.

■ The footer is the magic

Before reading any data, an engine reads the footer and decides **what it can skip**.

What's in it

- Schema (columns, types)
- Row-group locations
- **Min / max / null count** per column, per row group

What it enables

- **Projection pushdown** — jump straight to the columns you select
- **Predicate pushdown** — skip whole row groups that can't match
- Parallelism — hand each row group to a worker

- **Row-group size is a tuning knob**

Too small	Too large
metadata overhead explodes	fewer groups to skip (coarse pruning)
worse compression	less parallelism
many tiny pages	high memory per read

64–512 MB per row group is the production sweet spot — smaller for latency-sensitive lakes, larger for archival.



Part 4

Avro + Parquet together

Your guides to applied AI



■ Avro vs Parquet — and where each fits

Avro — row-oriented



record by record · write-optimized

streaming / ingest

schema evolution

Kafka

VS

Parquet — columnar



column by column · read-optimized

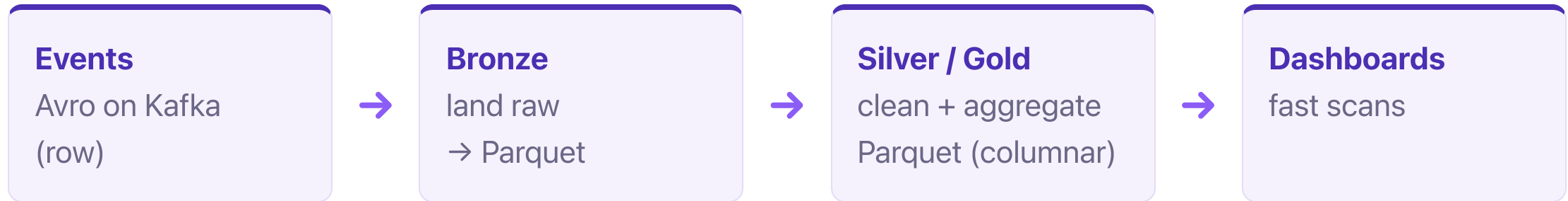
analytics / reporting

pushdown + stats

lakehouse

In our logistics pipeline: parcel scans arrive as **Avro** events on Kafka → land in **Bronze** → convert to **Parquet** for the reporting that feeds dashboards.

■ Row in, columnar for analytics



Why switch formats

- Ingest is **write-heavy, record-at-a-time** → Avro
- Analytics is **read-heavy, column-at-a-time** → Parquet
- Convert **once**, on the way into the lake

In the medallion

- **Bronze** ← raw rows land (often from Avro)
- **Silver / Gold** → columnar Parquet for queries
- One pipeline, the right format at each step



Part 5

Pushdown & partitioning

Your guides to applied AI



■ What is pushdown?

Pushdown = push the query's work *down* to the storage layer, so the reader **fetches less** instead of loading everything and filtering in memory.

Without pushdown

- Read **all columns, all rows** into memory
- Filter afterwards, inside the engine
- Wastes disk reads, memory and time

With pushdown

- Use `SELECT` + `WHERE` to **skip data on disk**
- Read only **needed columns + matching row groups**
- The Parquet **footer** (schema + min/max) makes it possible

Less data read = faster query, for free — just from how the file is laid out

■ Two kinds of pushdown

Projection — from `SELECT`

- `SELECT amount, order_ts ...`
- Reads **only those column chunks**
- Other columns are **never touched** on disk
- Pays off with **wide tables** (many columns)

Predicate — from `WHERE`

- `WHERE amount > 150`
- Footer says a group's **max is 120** → **skip it**
- No page in that group is read
- Pays off with a **selective filter**

Both happen inside a single file — before partitioning even enters the picture

■ Pushdown on a real query

```
SELECT country, amount FROM orders WHERE amount > 150
```

ON THE TABLE

id	country	amount
1	CZ	120
2	DE	80
3	CZ	200
4	FR	300

SELECT drops `id` · **WHERE** drops rows ≤ 150

HOW PARQUET RUNS IT

ROW GROUP 0 · AMOUNT MAX = 120

max < 150 → **skip the whole group**

ROW GROUP 1 · AMOUNT MAX = 300

read **country + amount** chunks · skip **id**

■ What is partitioning?

Partitioning physically splits one dataset into **many files, grouped by a column's value** — so a filter on that column can skip whole files without opening them.

The idea

- The value lives **in the file path**, not just inside the file
- The engine rules files out by **reading the directory**, not the data
- This is **partition pruning** — pushdown one level up

Hive-style layout

- Folders named `column=value/`
- The de-facto convention (Spark, Hive, DuckDB, Polars...)
- Stacks **on top of** row-group pushdown

■ Partitioning in practice

Hive-style partitioning splits the dataset into **folders named** `col=value/` :

```
orders_by_date/  
  order_date=2026-09-28/  data.parquet  
  order_date=2026-09-29/  data.parquet  
  order_date=2026-09-30/  data.parquet    ← a query for the 30th reads only this
```

- `WHERE order_date = DATE '2026-09-30'` → the engine **lists folders** and opens just one.
- Then **predicate pushdown** skips row groups *inside* that one file.

Two layers of skipping stack: **prune folders**, then **prune row groups**.

■ Choose the partition key well

Good keys

- **Low cardinality**, matches how you **filter**
- Date binned to **day / month** (not raw timestamp)
- Region, category, source system

The over-partitioning trap

- High-cardinality key (raw `order_ts` , `customer_id`) → **thousands of tiny files**
- Many small files run **slower** than a few big ones
- Fix: **bin to coarser granularity**

Aim for files around **128–512 MB**. Partition to *prune*, not to shard into dust.



Part 6

File format vs. table format

Your guides to applied AI



■ Parquet is just files

What Parquet gives you

- Columnar, compressed, self-describing
- Pushdown + statistics
- Read by everything (DuckDB, Polars, Spark...)

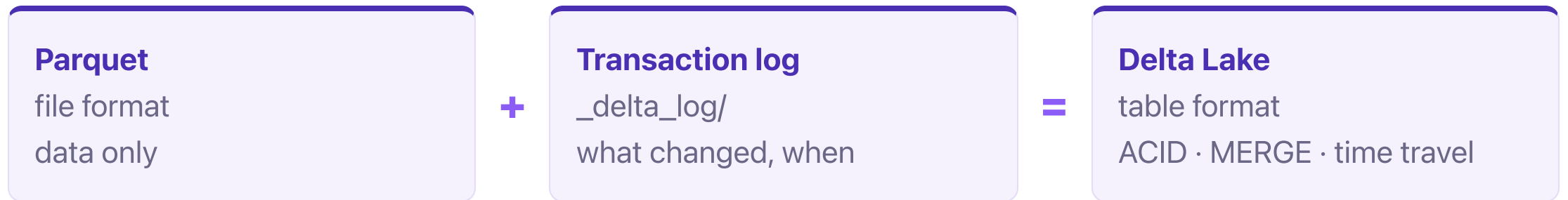
What it does **not** give you

- No **ACID transactions**
- No reliable **upsert / MERGE**
- No **history / time travel**
- No safe **concurrent writes**

ACID = Atomicity · Consistency · Isolation · Durability — the all-or-nothing guarantees a database gives a transaction.

A folder of Parquet files is a snapshot, not a table. Who's reading while you overwrite?

■ The road to Delta



- A **table format** wraps Parquet with a log → transactions, upserts, versions.
- Delta / Iceberg / Hudi all do this; same idea, different implementations.
- **Session 04:** we build the first Silver **Delta** table with version history.



Hands-on

Prove that pruning helps

Your guides to applied AI



■ Run it

```
git pull && uv sync
uv run eshop datagen && uv run eshop ingest # if you don't have Bronze yet
uv run python sessions/03-storage-formats/exercise/partition_lab.py
```

Then look at the physical layout partitioning creates:

```
ls data/lab/orders_by_date/ # one order_date=YYYY-MM-DD/ folder per day
```

On 20k orders the milliseconds are noisy — the point is *why* pruning helps, not the exact number.

■ Today's exercise

Fill in the `# TODO` s in `sessions/03-storage-formats/exercise/partition_lab.py` :

1. Write Bronze orders as a **Hive-partitioned** dataset (one folder per `order_date`).
2. Pick a day to filter on (`df["order_date"].max()`).
3. Complete the pruned query's **WHERE** so it filters on the **partition column**.
4. Write the data to **CSV** (comma-separated values) and compare **CSV vs. Parquet** file size.
5. **(harder)** Build **two-level** `year/month` partitioning and run a pruned month query.

■ The key lines

```
# Task 1 – partitioned write (Polars)
df.write_parquet(part_dir, partition_by=["order_date"])

# Task 3 – pruned read (DuckDB): filter on the partition column
con.execute(
    f"SELECT count(*) FROM read_parquet('{part}', hive_partitioning=true) "
    f"WHERE order_date = DATE '{target_day}'"
)

# Task 5 – two-level partitioning
df2.write_parquet(ym_dir, partition_by=["order_year", "order_month"])
```

Full scan filters inside one file; the pruned query skips whole folders first. CSV vs. Parquet shows the compression ratio.

■ Key takeaways

- **Row vs columnar** is about the *access pattern*: whole records (writes) vs a few columns over many rows (analytics).
- **Avro** — row-oriented, schema travels with the data, built for **streaming & schema evolution** (Kafka).
- **Parquet** — columnar: **row groups → column chunks → pages**, the **footer** carries statistics; compresses hard.
- **Projection + predicate pushdown** skip columns and row groups *before* reading data.
- **Partitioning** = pushdown at the folder level; prune to filter, but beware **over-partitioning / small files**. Parquet is a **file format** — a **table format** (Delta) adds ACID → next session.

■ Next time

Session 04 — Delta Lake: a table format on top of Parquet

- Why a Data Lake alone isn't enough: ACID, reliable upserts, history
- **MERGE** upsert and **time travel** over older versions
- The first **Silver Delta** table in our pipeline

■ References & further reading

- **Apache Parquet — file format spec** (magic number, row groups, footer) — parquet.apache.org/docs/file-format
- Row- vs column-oriented databases (ClickHouse) — clickhouse.com
- What is columnar storage? history + examples (MotherDuck) — motherduck.com
- Dremel: interactive analysis of web-scale datasets (Google, 2010) — research.google
- Apache Avro — object container file spec & schema evolution — avro.apache.org
- Avro vs Parquet — row vs columnar, when to use each (Starburst) — starburst.io
- Partition pruning & pros/cons of Hive partitioning — delta.io/blog
- Delta Lake (ACID table format) — delta.io

The background of the slide features a series of concentric, wavy lines in various shades of purple, creating a sense of depth and movement.

bighub

Thank you

Let's build.

www.bighub.ai

Your guides to applied AI

